TECHNICAL UNIVERSITY OF MOLDOVA
Faculty of Computers, Informatics, and Microelectronics
Department of Software Engineering
Software Engineering Study Programme

LABORATORY REPORT

Lab 3: The Command Line: Shell, Pipes and Text Filters

Operating Systems

Student: Gafenco Victor

Group: FAF-241

Instructor: Assistant lecturer, Patricia Reițman

Chișinău - 2026

CONTENTS

Abstract **3**

Introduction **4**

1 Command lookup and interactive assistance **5**

1.1 Completion and history 5

1.2 Built-ins, PATH and failure status 5

1.3 Finding help 6

2 Expansion and quoting **7**

2.1 Experimental files and predictions 7

2.2 Responsibility for constructing arguments 8

3 Streams and process control **9**

3.1 Separating output and diagnostics 9

3.2 The order of redirections 9

3.3 Input, chaining and asynchronous execution 9

4 Answering questions with pipelines **11**

4.1 Data model and consistency checks 11

4.2 Q1: failed requests 11

4.3 Q2: distinct pages 11

4.4 Q3: requests per page 11

4.5 Q4: users with the most failures 12

4.6 Q5: user3 12

4.7 Q6: the last failed records 12

4.8 Q7: configured account shells 12

4.9 Q8: the extra long-listing line 13

4.10 Regular-expression bonus 13

5 A reusable report program **14**

5.1 Implementation and measured behaviour 14

5.2 Interpreter, permissions and exit status 15

Conclusions **16**

ABSTRACT

This work examines the shell as an interpreter between a user and operating-system services. Experiments on openSUSE Tumbleweed establish how Bash resolves commands, expands arguments, connects streams and manages asynchronous jobs. A deterministic 500-record data set supports eight pipeline exercises and a reusable reporting script. Independent aggregation checks establish 100 failures and an even distribution of requests across four pages. The results also demonstrate why a maximum-count query must preserve ties and why redirection order changes observable behaviour.

Keywords: Bash, expansion, file descriptors, pipes, processes, text filters

INTRODUCTION

The purpose of the laboratory is to explain command-line behaviour through predictions and measured results, then combine small text-processing tools into a working program. The course handout provides five practical parts [**?**]. The Linux exercises were run in `~/os-lab3`; the report and implementation are retained in the existing course repository.

Commands use the C locale and Bash without personal startup files. Predictions were saved before the first expansion experiments. Figures show saved terminal output; the corresponding plain text is retained with the laboratory files.

1 COMMAND LOOKUP AND INTERACTIVE ASSISTANCE

1.1 Completion and history

Tab completed `/usr/sh` to the unique existing directory `/usr/share/`. The next Tab did not display competing alternatives in this environment. Completion of `/et` produced `/etc/`, and executing the completed `cd` command changed the working directory accordingly. Ctrl+R recovered `echo history_marker_lab3` after two intervening commands; Enter executed the recovered line. Completion and reverse search are shell editing facilities, rather than features of the commands being prepared.

```
LAB$ printf "Readline keys tested: ls /usr/sh<Tab><Tab>, cd /et<Tab><Enter>\n"
Readline keys tested: ls /usr/sh<Tab><Tab>, cd /et<Tab><Enter>
LAB$ pwd
/etc
LAB$ compgen -f /usr/sh
/usr/share
LAB$ compgen -d /et
/etc
LAB$ cd ~/os-lab3
```

Figure 1.1 – Completion targets and the resulting directory

1.2 Built-ins, PATH and failure status

Both `cd` and `echo` resolved to Bash built-ins. The `ls` executable resolved to `/usr/bin/ls`. A separate `cd` process could change only its own current directory; therefore the interactive shell must perform this operation in its own process. An external implementation of `echo` can coexist with the built-in without changing the lookup result observed here.

```
LAB$ type cd; type ls; type echo; which ls
cd is a shell builtin
ls is /usr/bin/ls
echo is a shell builtin
/usr/bin/ls
```

Figure 1.2 – Command classification and executable location

The clean-shell PATH check contains five entries: `~/.local/bin`, `/usr/local/bin`, `/usr/bin`, `/bin` and `/usr/sbin`. The count depends on the configured shell environment.

```
LAB$ echo "$PATH" | tr ':' '\n'
/home/victor/.local/bin
/usr/local/bin
/usr/bin
/bin
/usr/sbin
LAB$ printf '%s\n' "$PATH" | awk -F: '{print "PATH entries:", NF}'
PATH entries: 5
```

Figure 1.3 – PATH search locations and entry count

An unknown executable name produces status 127. The shell reports that it could not find a command; this differs from a program starting successfully and then returning an error of its own.

```
LAB$ nosuchprog; echo "exit code: $?"
bash: nosuchprog: command not found
exit code: 127
```

Figure 1.4 – Status returned for a command that does not exist

1.3 Finding help

The requested commands were `man ls`, `ls --help`, `man -k "list directory"` and `help cd`. The first manual lookup exposed a missing `ls` manual page on this installation. The corrected setup includes `coreutils-doc`, the separate package for these pages. A final lookup using the local manual search path succeeded and the manual index found `ls`, `dir` and `vdir`. The option `-t` sorts entries by modification time, newest first. Inside a manual pager, `/sort` searches, `n` repeats the search, and `q` exits. Bash's `help` documents its built-ins; `man -k` depends on the installed manual-page index.

2 EXPANSION AND QUOTING

2.1 Experimental files and predictions

The test directory initially contained two C filenames, three ordinary text filenames, and `alfa beta.txt`. Later, brace expansion created three log files.

```
LAB$ cd ~/os-lab3/play
LAB$ touch a.c b.c file1.txt file2.txt notes.txt "alfa beta.txt"
LAB$ ls
 a.c  'alfa beta.txt'   b.c   file1.txt   file2.txt   notes.txt
```

Figure 2.1 – Files used for expansion experiments

Command	Prior prediction	Observed output
`echo *.c`	a.c b.c	a.c b.c
`echo file?.txt`	file1.txt file2.txt	file1.txt file2.txt
`echo [a-b]*`	a.c alfa beta.txt b.c	Same names, same order
`echo *.xyz`	Literal *.xyz	Literal *.xyz
`ls alfa beta.txt`	Two missing paths	Errors for alfa and beta.txt
`ls "alfa beta.txt"`	One filename; possible display quotes	’alfa beta.txt’
`echo '$HOME' "$HOME" \$HOME`	Literal, expanded, literal	`$HOME /home/victor $HOME`
`echo {1..5}; echo $((7 * 6))`	1 2 3 4 5; then 42	As predicted
`echo "today is $(date +%A), I am $(whoami)"`	Runtime weekday and login	today is Monday, I am victor
`touch log{1..3}.txt && ls log*`	Three numbered log names	log1.txt log2.txt log3.txt

The recorded predictions agreed with the semantic results. Terminal column spacing and GNU `ls`’s presentation quotes do not change the underlying filenames. The unmatched pattern stayed literal because default Bash glob options were used.

```
LAB$ echo *.c
a.c b.c
LAB$ echo file?.txt
file1.txt file2.txt
LAB$ echo [a-b]*
a.c alfa beta.txt b.c
LAB$ echo *.xyz
*.xyz
```

Figure 2.2 – Matched and unmatched pathname patterns

2.2 Responsibility for constructing arguments

The shell expands `*.c` before it invokes `echo`; the receiving program need not enumerate a directory. A glob result containing a space remains one argument, even though the displayed output alone does not show that boundary. In contrast, unquoted spaces in the command text split `alfa beta.txt` into two arguments. Either double quotes or a backslash before the space fixes the call.

```
LAB$ ls alfa beta.txt
ls: cannot access 'alfa': No such file or directory
ls: cannot access 'beta.txt': No such file or directory
LAB$ ls "alfa beta.txt"
'alfa beta.txt'
LAB$ ls alfa\ beta.txt
'alfa beta.txt'
```

Figure 2.3 – Failure caused by unquoted whitespace and two fixes

Single quotes inhibit expansion. Double quotes preserve word boundaries while allowing variable, arithmetic and command substitution. An escaped dollar sign is literal. GNU `ls` may add quotes when displaying a spaced filename in a terminal; these are presentation characters, not part of the stored name.

```
LAB$ echo '$HOME' "$HOME" \$HOME
$HOME /home/victor $HOME
LAB$ echo {1..5}; echo $((7 * 6))
1 2 3 4 5
42
LAB$ echo "today is $(date +%A), I am $(whoami)"
today is Monday, I am victor
LAB$ touch log{1..3}.txt && ls log*
log1.txt  log2.txt  log3.txt
```

Figure 2.4 – Quoting, arithmetic, brace and command expansion

3 STREAMS AND PROCESS CONTROL

3.1 Separating output and diagnostics

In `ls /etc /nope > out.txt 2> err.txt`, descriptor 1 carries the directory listing into `out.txt`, while descriptor 2 carries the missing-path diagnostic into `err.txt`. Redirecting both streams to one file does not guarantee that an error will appear at its end: buffering and write order still determine placement.

```
LAB$ cd ~/os-lab3
LAB$ ls /etc /nope > out.txt 2> err.txt
LAB$ wc -l out.txt; cat err.txt
214 out.txt
ls: cannot access '/nope': No such file or directory
LAB$ ls /etc /nope > all.txt 2>&1; tail -2 all.txt
xscreensaver
zypp
```

Figure 3.1 – Directory output and error output in separate files

3.2 The order of redirections

Redirection is processed from left to right. With `> all.txt 2>&1`, stdout first changes destination and stderr then duplicates that destination. With `2>&1 > all.txt`, stderr retains the original terminal destination while stdout is subsequently moved to the file. The experiment therefore makes the second version's error visible on the terminal.

```
LAB$ ls /etc /nope > all.txt 2>&1
LAB$ head -3 all.txt
ls: cannot access '/nope': No such file or directory
/etc:
IM-7
LAB$ ls /etc /nope 2>&1 > all.txt
ls: cannot access '/nope': No such file or directory
LAB$ head -3 all.txt
/etc:
IM-7
NetworkManager
```

Figure 3.2 – Different outcomes from reversing redirection order

3.3 Input, chaining and asynchronous execution

An input redirection gives `wc` bytes on standard input without a pathname argument, so `wc -l < access.log` prints only 500. Supplying the pathname as an argument additionally prints its name. Truncating and appending writes to `notes.md` preserve the two lines `first` and `second`.

A semicolon sequences commands irrespective of status. The `&&` operator continues only after success; `||` continues only after failure. The failing GNU `ls /nope` returns 2, which the following conditional

message reports.

```
LAB$ echo first > notes.md; echo second >> notes.md; cat notes.md
first
second
LAB$ wc -l < access.log
500
LAB$ wc -l access.log
500 access.log
LAB$ ls /nope 2> /dev/null || echo "failed with code $?"
failed with code 2
LAB$ mkdir -p tmp && cd tmp && echo "I am in $(pwd)"; cd ~/os-lab3
I am in /home/victor/os-lab3/tmp
```

Figure 3.3 – Input redirection, appending and conditional commands

The background `sleep 100` appears as job 1 and is signalled by `kill %1`. An immediate job listing may still show Running; a later check observes termination. This illustrates asynchronous delivery and collection of process status rather than a failed kill command.

```
LAB$ sleep 100 &
[1] 659208
LAB$ jobs; kill %1; jobs
[1]+  Running                 sleep 100 &
[1]+  Running                 sleep 100 &
LAB$ sleep 0.2; jobs
[1]+  Terminated              sleep 100
```

Figure 3.4 – Starting, inspecting and terminating a background job

4 ANSWERING QUESTIONS WITH PIPELINES

4.1 Data model and consistency checks

The prescribed generator emits 500 records with date, time, user, result and page fields. Every fifth index fails, every fourth selects the same page, and users cycle modulo seven. Queries filter the relevant fields rather than matching unrelated substrings. Each of Q1–Q8 occupies one executable line in `oneliners.sh`; comments identify the question.

4.2 Q1: failed requests

The measured failure count is 100, consistent with one failure per five records.

```
LAB$ awk '$4 == "FAIL" {n++} END {print n+0}' access.log
100
```

Figure 4.1 – Counting failed requests

4.3 Q2: distinct pages

Sorting the fifth field and removing duplicates yields four paths, from `/page0` through `/page3`.

```
LAB$ cut -d' ' -f5 access.log | sort -u | awk '{print} END {print "pages:", NR}'
/page0
/page1
/page2
/page3
pages: 4
```

Figure 4.2 – The four distinct page names

4.4 Q3: requests per page

Each page receives 125 requests. Their sum is 500, independently matching the number of generated records.

```
LAB$ cut -d' ' -f5 access.log | sort | uniq -c
    125 /page0
    125 /page1
    125 /page2
    125 /page3
```

Figure 4.3 – Equal request counts across the pages

4.5 Q4: users with the most failures

Both user3 and user5 have 15 failed requests. Every other user has 14. The query retains every count equal to the first, maximum count after descending sorting; using only `head -1` would incorrectly discard a tie. The seven counts sum to 100, independently agreeing with Q1.

```
LAB$ awk '$4 == "FAIL" {print $3}' access.log | sort | uniq -c | sort -k1,1nr -k2,2 | awk 'NR == 1
{max=$1} $1 == max'
     15 user3
     15 user5
```

Figure 4.4 – Both users sharing the maximum failure count

4.6 Q5: user3

Filtering the third field yields 72 requests for user3, of which 15 failed.

```
LAB$ awk '$3 == "user3" {n++; if ($4 == "FAIL") f++} END {print "user3:", n+0, "requests,", f+0,
"failed"}' access.log
user3: 72 requests, 15 failed
```

Figure 4.5 – Total and failed requests for user3

4.7 Q6: the last failed records

The final three matching rows yield `10:10 user0`, `10:15 user5`, and `10:20 user3`. Here last refers to file order: the generated minutes cycle and must not be mistaken for a monotonically increasing timestamp.

```
LAB$ awk '$4 == "FAIL" {print $2, $3}' access.log | tail -3
10:10 user0
10:15 user5
10:20 user3
```

Figure 4.6 – Time and user from the last three failed records

4.8 Q7: configured account shells

The observed account file contains two entries with `/bin/bash` and 32 with `/usr/sbin/nologin`. These are account configurations, including service accounts, rather than a count of current interactive users.

```
LAB$ cut -d: -f7 /etc/passwd | sort | uniq -c
      2 /bin/bash
     32 /usr/sbin/nologin
```

Figure 4.7 – Counts grouped by the account shell field

4.9 Q8: the extra long-listing line

The prior prediction was a difference of one. The two pipelines returned 213 and 214: ordinary piped `ls` emits one name per line, whereas its long form adds a `total` header. These exact counts are specific to this installation. Embedded newlines in filenames, aliases or concurrent changes would require additional care when generalising the result.

```
LAB$ ls /etc | wc -l; ls -l /etc | wc -l
213
214
```

Figure 4.8 – The additional header in a long directory listing

```
LAB$ bash oneliners.sh
100
/page0
/page1
/page2
/page3
pages: 4
    125 /page0
    125 /page1
    125 /page2
    125 /page3
     15 user3
     15 user5
user3: 72 requests, 15 failed
10:10 user0
10:15 user5
10:20 user3
      2 /bin/bash
     32 /usr/sbin/nologin
213
214
```

Figure 4.9 – Executing all eight one-liners in order

4.10 Regular-expression bonus

A `grep -E` expression restricts the time to `10:[0-5]0` and the result field to FAIL. It prints 50 records. Each tenth generator index satisfies the minute condition and is also divisible by five. The executable query is retained separately as `bonus.sh`.

5 A REUSABLE REPORT PROGRAM

5.1 Implementation and measured behaviour

The completed script checks the argument count and file readability, obtains the total and failed counts, then iterates over user0 through user6. Field-aware aggregation returns explicit zeroes for empty sets. Quoted path arguments preserve filenames containing spaces.

```
LAB$ cat report.sh
#!/bin/bash
# usage: ./report.sh access.log
if [ "$#" -ne 1 ]; then
    echo "usage: $0 access.log" >&2
    exit 1
fi
log=$1
if [ ! -f "$log" ] || [ ! -r "$log" ]; then
    echo "no such readable file: $log" >&2
    exit 1
fi
total=$(wc -l < "$log")
fails=$(awk '$4 == "FAIL" {n++} END {print n+0}' "$log")
echo "$total requests, $fails failed"
for user in user{0..6}; do
    count=$(awk -v user="$user" '$3 == user && $4 == "FAIL" {n++} END {print n+0}' "$log")
    echo "$user: $count failures"
done
```

Figure 5.1 – Implementation of the reporting script

On the shared data it reports 500 requests and 100 failures, followed by counts 14, 14, 14, 15, 14, 15 and 14 for the seven users.

```
LAB$ chmod +x report.sh
LAB$ ./report.sh access.log
500 requests, 100 failed
user0: 14 failures
user1: 14 failures
user2: 14 failures
user3: 15 failures
user4: 14 failures
user5: 15 failures
user6: 14 failures
```

Figure 5.2 – Successful report for the generated log

A nonexistent pathname produces an error on descriptor 2 and status 1. Missing or extra arguments also fail. Additional checks exercised an empty input, a zero-failure input and a pathname containing spaces.

```
LAB$ ./report.sh nope.log; echo "exit code: $?"
no such readable file: nope.log
exit code: 1
LAB$ ./report.sh; echo "exit code: $?"
usage: ./report.sh access.log
exit code: 1
```

Figure 5.3 – Missing-file and missing-argument diagnostics

5.2 Interpreter, permissions and exit status

The shebang `##!/bin/bash` identifies the interpreter for direct execution. Without execute permission, `./report.sh` is rejected, normally with status 126; explicitly running `bash report.sh` instead requires read permission. The `./` prefix selects a pathname in the current directory rather than relying on a PATH entry. Diagnostic output on stderr preserves the normal data stream, and status 1 permits a caller to react to failure.

The optional cron bonus was not executed; no scheduled-run evidence is claimed. The notebook includes a full-path example for that optional extension. Unrelated crontab entries must be retained when adding or removing a temporary lab schedule.

CONCLUSIONS

The experiments identify three responsibilities that are easy to confuse: Bash constructs arguments, redirections construct descriptor connections, and programs consume the resulting arguments and streams. Small utilities become useful data analysis tools when their inputs and fields are well defined. The agreement of independent totals validates the pipelines, while the tied failure counts show why seemingly minor query choices affect correctness. The final script makes these observations reusable through explicit validation and meaningful status.